SQ-DiT: SliderQuant for Diffusion Models

August 28, 2026

Simone Lidonnici - 2061343 Pietro Adamo Morelli - 2087359

Abstract

Diffusion Transformers (DiTs) have recently established a new baseline for image generation models, but their major limitations are the heavy computational and memory requirements. To mitigate these bottlenecks, Post-Training Quantization (PTQ) offers a practical approach to compress these models without retraining. In this project, we adapt SliderQuant, a PTQ algorithm originally designed for Large Language Models (LLMs), to the visual generation domain. Code is available at: https://github.com/DoctorWho28/deepLearning.

1. Introduction

Diffusion Transformers (DiT) created a new standard of generative AI, replacing the convolutional U-Net backbone of diffusion models with a Transformer architecture. DiTs rely on **Adaptive Layer Normalization (AdaLN)** to inject continuous timestep and discrete class conditionings into each block, dynamically adapting intermediate feature states at each denoising step. DiTs achieve remarkable results in high-fidelity image generation, but their large parameter count and iterative approach lead to prohibitive memory consumption (VRAM) and slow inference speeds. To mitigate these computational bottlenecks, Post-Training Quantization (PTQ) techniques compress the weights to lower-bit formats without requiring a full model retraining.

Recently, a PTQ framework named **SliderQuant** was introduced for Large Language Models (LLMs). SliderQuant exploits the fact that different layers in deep networks have different sensitivities to quantization noise. It introduces a sliding-window distillation approach, optimizing quantization parameters over small groups of layers, and introduces learnable low-rank matrices (LoRA) to compensate for the quantization error.

Email:
Simone Lidonnici <lidonnici.2061343@studenti.uniroma1.it>,
Pietro Adamo Morelli <morelli.2087359@studenti.uniroma1.it>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

SQ-DiT aims to adapt the core concepts of SliderQuant to Diffusion Transformers. The main difference is that SliderQuant calibrates LLMs over sequential tokens, whereas DiTs require optimizing across the temporal diffusion process. During the optimization process, the weights are quantized using fake quantization and only upon completion the final weights are packed into INT8 using our custom packing implementation to effectively save VRAM space.

Aggiungere la specificazione sui modelli ImageNet-1k class-to-image e scrivere del modello facebook/DiT-XL-2-256

2. Related works

SQ-DiT derives from SliderQuant (Wang et al., 2026) adapting the quantization and distillation concepts to the DiT architecture.

PTQ4DiT (Wu et al., 2024) and Q-DiT (Chen et al., 2024) serve as our primary foundational references for the state of the art in PTQ for DiTs.

Possibile escursus sui paper usati per confrontare i risultati

3. Method

3.1. SliderQuant

SliderQuant builds on the premise that initial (shallow) and terminal (deep) layers have higher quantization sensitivity than intermediate ones. These boundary layers are optimized using a progressively expanding window, while the intermediate layers are optimized using a fixed size sliding-window. In both cases, layers in the same window are quantized jointly.

To minimize the weight quantization error, an optimizer is applied to a set of learnable parameter for each linear module, specifically a scaling factor (α) and two low-rank LoRA matrices (A and B).

To soften the impact on the weights and avoid excessive parameter perturbations, quantization is performed in two progressive stages: a partial quantization is applied according to a ratio γ, followed by the full quantization. To mitigate the impact of extreme weight values, we apply group-wise quantization, confining outliers to local groups and preserving the precision of the remaining parameters.

3.2. SQ-DiT implementation

Because the original SliderQuant framework cannot be easily transposed, we developed the entire quantization framework from scratch.

Timestep-Aware Window Distillation. We group transformer blocks into sliding windows and optimize them sequentially. For each window, we optimize the quantization parameters by minimizing the Mean Squared Error (MSE) between the output features of the quantized blocks with their original unquantized counterparts. SliderQuant uses the L^2 norm instead of the MSE.

Unlike LLMs, where the input passes through each block only once, in DiT models the image is gradually denoised across multiple timesteps, passing through all the blocks at each step.

Since data distributions change drastically throughout the denoising process, a quantization approach that propagates error across the timesteps would be unstable. For this reason, rather than allowing the error to propagate over time, each window jointly optimizes multiple timesteps. We provide the first block with the latents pre-calculated by the original FP16 model across all timesteps, while the subsequent blocks receive the output of the preceding quantized block.

By eliminating temporal propagation, we align the optimization with the SliderQuant paradigm used for LLMs, that don't have noise accumulation between timesteps. This allows the LoRA matrices within each window to focus exclusively on compensating for spatial errors propagated by preceding quantized layers.

Mixed-Precision Architecture. Following the original observation that network layers exhibit varying sensitivities to quantization, we decided to augment the SliderQuant approach by applying a mixed-precision policy. Deep and shallow transformer blocks are highly sensitive and are therefore quantized to higher bit-widths (INT8 in our case) while intermediate layers, which are more robust to quantization noise, can be quantized to a lower bit-widths.

Other than that, crucial modules responsible for condition injection require special treatment. The initial `emb` module, which maps discrete time and class conditions into a continuous embedding space, is kept in FP16. Since this conditioning guides the entire generative process, any error introduced here would cascade through all subsequent layers and degrade the feature representations. Similarly, the AdaLN (`norm1`) module, responsible for injecting temporal and class features to the network, is quantized to higher precision. Aggressively quantizing this layer would cause the network to lose temporal awareness and class concepts, leading to generation failures.

Using this mixed-precision approach has a slight impact on VRAM utilization, but this is negligible compared to the benefits on generation stability.

Weights Packing. To effectively have an impact on the model size and VRAM utilization we implemented a custom `WXAXLinear` module as a replacement for standard linear layers. This module physically packs low-bit quantized weights into `INT8` using bitwise shift and masking operations.

When loading the model to generate an image, we replace the original linear layers with their `WXAXLinear` counterparts and load the packed weights into memory. During inference, the layer performs on the fly unpacking, restoring the weights to FP16 dynamically to continue the image generation process.

4. Evaluation

Sensitivity Analysis Following the principle introduced by SliderQuant, we conducted a sensitivity analysis to determine the optimal number of deep and shallow layers for DiTs. Our sensitivity evaluation methodology is inspired by the Sensitivity-Aware Mixed Precision Quantization (Huang & Badaoui, 2025) for LLMs, which estimates layer sensitivity by quantizing a single layer at a time, while keeping the rest of the network in FP16. In the original approach, divergence is measured using the Jensen-Shannon Divergence (JSD) between output probability distributions. However, since DiTs output continuous values rather than probabilities, JSD is inapplicable. We replaced it with the linear inverse of the Signal-to-Noise Ratio (SNR) (Roux et al., 2018) to evaluate the continuous error.

Optimization Parameters The training was done using a *GPU Nvidia RTX 4090* and based on its computational capabilities we selected the hyperparameters to balance the computational time required and the quantization quality.

The optimization was conducted for **30 epochs** per quantization stage (60 in total), which proved sufficient for the learnable parameters to converge and effectively compensate for the quantization noise, avoiding overfitting. To ensure that the calibrated parameters generalize well across different image concepts, we uniformly sampled a subset of **20 classes**, taking one every 50 classes, for our calibration. Furthermore, we optimized the model over **20 inference steps**, which we found to be sufficient to generate images of acceptable visual quality.

Scaling up the calibration process by including more classes or inference timesteps would have been prohibitively expensive in terms of computational time and

(a) Original (b) W8A8 (c) W4A8

Figure 1. Visual comparison of generated samples

hardware costs.

Quantitative evaluation To evaluate the generative quality of our quantized models against the original, we used several standard quantitative metrics, specifically **Fréchet Inception Distance (FID)**, **spatial FID (sFID)**, **Inception Score (IS)**.

Additionally, we utilized **CMMD** (CLIP Maximum Mean Discrepancy) (Jayasumana et al., 2024), a modern and highly sample-efficient metric, which provides a robust assessment of image quality.

Table 1 presents the evaluation of our DiT models with a W8A8 (8-bit weights and activations) and W4A8 (4-bit weights and 8-bit activations) configurations. The **model size** is reported to quantify the benefits in VRAM utilization achieved by our INT8 packing mechanism.

vere che
ntrambi i
l'higher
ision è 8

Table 1. Evaluation comparison

	Size	FID ↓	sFID ↓	IS ↑	CMMD ↓
FP16	1.58 GB	24.72	59.3	318 ± 7.9	**0.236**
W8A8	0.97 GB	24.48	57.8	**322 ± 6.0**	0.239
W4A8	0.78 GB	**23.49**	**55.7**	314 ± 6.7	**0.236**

Our quantization framework achieves significant memory reductions while fully preserving generative quality. Moving from the FP16 baseline to W8A8 reduces the model size by roughly 38%, and the W4A8 configuration effectively halves it. We observe a numerical improvement in the FID and sFID scores for the quantized models, this behavior is a known artifact of how these metrics measure distribution distances, as they can sometimes reward the slight smoothing effect introduced by quantization. For a more reliable assessment of generative fidelity, we look at the CMMD metric which remains stable across all models, demonstrating that quantization preserves the visual integrity as shown in Figure 1.

5. Conclusions

Statement on the use of AI

Students must include a brief statement describing whether AI tools were used in the project and, if so, how they were used. This declaration is required for transparency, but it is not intended to penalize the use of AI in any way. A project may receive any grade in the full $0-31$ range regardless of whether AI tools were used extensively, minimally, or not at all.

The statement should clearly indicate the role of AI in the project. Examples include, but are not limited to: rephrasing or editing parts of the report, assisting with code writing, implementing core algorithms, preparing experiment scripts, debugging, data processing, literature exploration, or generating plots and other supporting material. Students are expected to be honest and sufficiently specific.

The responsibility for the submitted work always remains with the student(s). In particular, students must fully understand, verify, and be able to explain any material produced with the help of AI tools.

Misuse of AI will be penalized. Examples include submitting AI-generated material that the student does not understand, cannot justify, or cannot explain if required to do so; presenting AI-produced text, code, experimental results, or claims as if they had been independently developed or verified when this is not the case; or providing inaccurate or misleading declarations about the actual use of AI in the project.

There is no strict length limit for this section, but in most cases one column or less should be sufficient.

References

Chen, L., Meng, Y., Tang, C., Ma, X., Jiang, J., Wang, X., Wang, Z., and Zhu, W. Q-dit: Accurate post-training quantization for diffusion transformers, 2024. URL https://arxiv.org/abs/2406.17343.

Huang, J. and Badaoui, A. Sensitivity aware mixed precision quantization v1, 2025. URL https://huggingface.co/blog/badaoui/sensitivity-aware-mixed-precision-quantizer-v1.

Jayasumana, S., Ramalingam, S., Veit, A., Glasner, D., Chakrabarti, A., and Kumar, S. Rethinking fid: Towards a better evaluation metric for image generation, 2024. URL https://arxiv.org/abs/2401.09603.

Roux, J. L., Wisdom, S., Erdogan, H., and Hershey, J. R. Sdr - half-baked or well done?, 2018. URL https://arxiv.org/abs/1811.02508.

Wang, S., Li, C., Kang, Y., Fan, J., Ou, Z., and Yao, A. Sliderquant: Accurate post-training quantization for llms, 2026. URL https://arxiv.org/abs/2603.25284.

Wu, J., Wang, H., Shang, Y., Shah, M., and Yan, Y. Ptq4dit: Post-training quantization for diffusion transformers, 2024. URL https://arxiv.org/abs/2405.16005.

Appendix

Sensitivity analysis

$$NSR_l = 10^{\frac{-SNR_l}{10}}$$

$$Score_l = NSR_l \cdot \left(1 + \lambda \cdot \frac{magnitude_l}{\max_k magnitude_k}\right)$$

Table 2. Hyperparameters

Configuration	Setting
Epoch	30
Classes	20
Inference timesteps	20
Batch size	4
Layer shallow	3
Layer deep	2
Window size	2
Window step	1
γ	0.5
Rank of LoRA	16
Quantization group size	128

Simulation parameters

Final weights formula

$$W = W \cdot \alpha + A \times B$$